ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP 2

MÔN: CÁC HỆ CƠ SỞ TRI THỨC

BÀI 1

BIỂU DIỄN TRI THỨC TAM GIÁC BẰNG MẠNG TÍNH TOÁN

Giảng viên: PGS.TS Nguyễn Đình Hiển

Sinh viên thực hiện:

Đoàn Hữu Gia Bình — 24520192

Nguyễn Thái Bảo — 24520173

Bùi Huy Chương — 24520230

Võ Hoài Chiều — 24520220

Thành phố Hồ Chí Minh, tháng 10 năm 2026

THÔNG TIN NHÓM VÀ PHÂN CÔNG CÔNG VIỆC

Môn học: Các hệ cơ sở tri thức (CS217). Giảng viên: PGS.TS Nguyễn Đình Hiển. Nhóm thực hiện Bài tập 2 gồm bốn thành viên; phân công giữa hai sản phẩm được thống nhất như sau.

Bảng 0.1. Phân công công việc của nhóm

Họ và tên	MSSV	Nội dung công việc
Đoàn Hữu Gia Bình	24520192	Lập trình, kiểm thử và hoàn thiện Bài 1
Nguyễn Thái Bảo	24520173	Xây dựng tài liệu và báo cáo Bài 1
Bùi Huy Chương	24520230	Lập trình, kiểm thử và hoàn thiện Bài 2
Võ Hoài Chiều	24520220	Xây dựng tài liệu và báo cáo Bài 2

Báo cáo này trình bày Bài 1. Nội dung được đối chiếu với mã nguồn tại commit 2ae25f2 và kiểm thử lại ngày 24/10/2026 trên Windows, Python 3.13.5. Các tệp báo cáo và ảnh bổ sung không làm thay đổi thuật toán hoặc dữ liệu CSTT.

Quy ước trình bày: chương đánh số 1–6; mục dùng dạng chương.mục; hình, bảng và công thức đánh số riêng theo chương. Các bảng ở phần mở đầu dùng tiền tố 0; phụ lục dùng chữ cái. Trích dẫn đặt trong ngoặc vuông và đối chiếu danh mục tài liệu tham khảo.

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN BÀI TOÁN....4
1.1. Bối cảnh và mục tiêu....4
1.2. Đầu vào, đầu ra và phạm vi....4
1.3. Ví dụ định hướng....4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT....5
2.1. Mạng tính toán và luật có hướng....5
2.2. Tập biến, miền và đơn vị....5
2.3. Các nhóm quan hệ....6
2.4. Ràng buộc và tính nhất quán....6
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG....7
3.1. Kiến trúc và trách nhiệm....7
3.2. Thuật toán suy diễn tiến....8
3.3. Vết suy luận và giải thích phụ thuộc....8
3.4. Hợp đồng kết quả và giao diện....9
CHƯƠNG 4. CÀI ĐẶT VÀ TRIỂN KHAI....10
4.1. Công nghệ và cấu trúc mã....10
4.2. Cài đặt và chạy local....10
4.3. CLI, REST API và thao tác sử dụng....11
CHƯƠNG 5. KIỂM THỬ VÀ KẾT QUẢ THỰC NGHIỆM....12
5.1. Môi trường và phương pháp kiểm tra....12
5.2. Đối chiếu T01–T12....12
5.3. Minh họa ứng dụng thực tế....14
5.4. Đánh giá và giới hạn thực nghiệm....14
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN....15
6.1. Kết quả đạt được....15
6.2. Hạn chế....15
6.3. Hướng phát triển....15
TÀI LIỆU THAM KHẢO....16
PHỤ LỤC A. DANH MỤC 46 LUẬT CÓ HƯỚNG....17

CHƯƠNG 1. TỔNG QUAN BÀI TOÁN

1.1. Bối cảnh và mục tiêu

Kiến thức tam giác gồm nhiều đại lượng liên hệ qua công thức. Một chương trình chỉ gắn mỗi công thức với một màn hình nhập liệu sẽ khó tự kết hợp nhiều quan hệ khi dữ kiện và mục tiêu thay đổi. Bài 1 tổ chức các công thức thành mạng tính toán (M, F), dùng suy diễn tiến để tìm các giá trị chưa biết và giải thích kết quả bằng chuỗi luật đã thực sự áp dụng [1], [2].

Mục tiêu là xây dựng một CSTT hữu hạn, tách phần mô tả luật khỏi cơ chế suy luận; kiểm tra miền và mâu thuẫn dữ kiện; cung cấp CLI, API và giao diện trực quan. Sản phẩm hiện có là chương trình Python/FastAPI kết hợp React, không còn chỉ là đề xuất thiết kế.

1.2. Đầu vào, đầu ra và phạm vi

Đầu vào H là ánh xạ từ tên biến đến giá trị số, G là tập đại lượng cần tìm. Đầu ra gồm trạng thái, các dữ kiện được chấp nhận, mục tiêu còn thiếu và vết suy luận. Người dùng không phải nhập đủ 13 biến; hệ thống chỉ áp dụng những quan hệ đã đủ dữ kiện.

Phạm vi tri thức được cố định ở 13 biến, 10 nhóm quan hệ Q01–Q10 và 46 luật R01–R46. Chương trình không phải bộ giải đại số tổng quát, không tự biến đổi công thức theo mọi chiều và không tự chọn nghiệm góc trong bài toán SSA có thể có hai nghiệm.

1.3. Ví dụ định hướng

Với a = 3, b = 4, c = 5, mục tiêu p, S, hc, r, R lần lượt có giá trị 6; 6; 2,4; 1; 2,5. Đây là bộ T01 được chạy lại bằng SolverService. Với a = 1, b = 2, c = 3, hệ thống từ chối bộ cạnh suy biến; với cùng bộ cạnh 3–4–5 nhưng S = 7, hệ thống phát hiện mâu thuẫn và giữ nguyên dữ kiện gốc.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Mạng tính toán và luật có hướng

$$r: u(r) \rightarrow \{y\};\ u(r) \text{ nằm trong } M;\ y \text{ thuộc } M \text{ trừ } u(r) \quad (2.1)$$

Mỗi luật r xác định một đầu ra y từ tập đầu vào u(r). Trong bài này, mỗi luật được cài bằng một callable tin cậy; chuỗi công thức chỉ dùng để trình bày [1], [2].

Quan hệ gốc và luật có hướng là hai mức biểu diễn khác nhau. Chẳng hạn A + B + C = 180° sinh ba luật tính từng góc từ hai góc còn lại. Ngược lại, định lý sin không được suy ngược bằng arcsin trong CSTT vì một giá trị sin có thể tương ứng hai góc trong khoảng (0°, 180°).

2.2. Tập biến, miền và đơn vị

$$M = \{a, b, c, A, B, C, ha, hb, hc, S, p, r, R\} \quad (2.2)$$

Các tên phân biệt chữ hoa và chữ thường. a, b, c đối diện lần lượt với A, B, C; p là nửa chu vi, r và R là bán kính nội tiếp và ngoại tiếp.

Bảng 2.1. Các đại lượng trong CSTT

Biến	Ý nghĩa	Miền và đơn vị
a, b, c	Ba cạnh tam giác	Số hữu hạn > 0, cùng đơn vị độ dài
A, B, C	Ba góc trong	0 < góc < 180, đơn vị độ
ha, hb, hc	Ba đường cao tương ứng	Số hữu hạn > 0, đơn vị độ dài
S	Diện tích	Số hữu hạn > 0, bình phương đơn vị độ dài
p	Nửa chu vi	Số hữu hạn > 0, đơn vị độ dài
r, R	Bán kính nội tiếp, ngoại tiếp	Số hữu hạn > 0, đơn vị độ dài

Góc nhập/xuất theo độ. Các hàm sin_deg, cos_deg và acos_deg chuyển đổi nhất quán với radian khi tính bằng Python. Việc làm tròn chỉ phục vụ hiển thị; suy luận và kiểm tra nhất quán sử dụng giá trị chưa làm tròn.

2.3. Các nhóm quan hệ

Bảng 2.2. Quan hệ gốc và số luật đã cài đặt

Nhóm	Quan hệ	Số luật
Q01	$A + B + C = 180°$	3
Q02	$2p = a + b + c$	4
Q03	$a/\sin A = b/\sin B = c/\sin C$	6
Q04	$a^2 = b^2 + c^2 - 2bc \cos A$; hoán vị	6
Q05	$S = \sqrt{[p(p-a)(p-b)(p-c)]}$	1
Q06	$2S = a \cdot ha = b \cdot hb = c \cdot hc$	9
Q07	$2S = bc \sin A$; hoán vị	3
Q08	$S = pr$	3
Q09	$4RS = abc$	5
Q10	$a = 2R \sin A$; hoán vị	6

Tổng số luật là 46. Q04, Q07 và Q10 bao gồm những hoán vị tương ứng; chỉ các chiều liệt kê trong Phụ lục A thuộc tập F. Ví dụ Q06 sinh R27: {c, hc} → S, R28: {S, c} → hc và R29: {S, hc} → c. Các cạnh của mạng thể hiện phụ thuộc tính toán, không khẳng định mọi quan hệ đều khả nghịch.

2.4. Ràng buộc và tính nhất quán

$$a + b > c;\ \ a + c > b;\ \ b + c > a \qquad (2.3)$$

Ba bất đẳng thức là ràng buộc hợp lệ của tam giác không suy biến, không phải ba luật mới làm tăng danh mục 46 luật. Validator sắp xếp $x \le y \le z$ rồi kiểm tra $x > z - y$, tương đương kiểm tra cạnh lớn nhất và tránh tràn số ở phép cộng [3].

Ngoài miền từng biến, hệ thống kiểm tra tổng hai góc nhỏ hơn 180°, tổng ba góc bằng 180° trong ngưỡng; p phải lớn hơn mỗi cạnh đã biết; đối chiếu nửa chu vi, Heron và mọi luật đã có đủ đầu vào lẫn đầu ra. Khi đủ ba cạnh và S nhưng chưa có p, diện tích vẫn được kiểm tra bằng Heron.

$$|x - y| \le \max(\varepsilon_{abs}, \varepsilon_{rel} \cdot \max(|x|, |y|)) \qquad (2.4)$$

Sai số tuyệt đối $\varepsilon_{abs} = 10^{-8}$ và tương đối $\varepsilon_{rel} = 10^{-7}$. Các phép chia, căn và phép tính góc phải cho kết quả hữu hạn thuộc miền. Công thức cosin và Heron được viết dưới dạng tương đương ổn định hơn về số học; chiều tính và tập đầu vào của luật không thay đổi.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc và trách nhiệm

Bảng 3.1. Các lớp chức năng của Bài 1

Thành phần	Trách nhiệm và kết nối
Models	Variable, Rule, SolveRequest, InferenceStep, SolveResponse và SolveStatus; kiểm tra cấu trúc bằng Pydantic.
Knowledge	RuleRegistry lưu metadata bất biến và callable riêng; 46 luật được quét theo thứ tự ID.
Core	InferenceEngine điều phối suy diễn; TriangleValidator kiểm tra miền và mâu thuẫn.
Services	SolverService là điểm gọi dùng chung cho CLI và FastAPI; hỗ trợ lọc trace phụ thuộc.
API/CLI	Nhận H, G và trả cùng kết quả nghiệp vụ; create_app(service) hỗ trợ tiêm service khi kiểm thử.
Frontend	React nhập dữ kiện, chọn mục tiêu, gọi API, trình bày kết quả và dựng tam giác SVG.

Luồng xử lý là CLI hoặc FastAPI → SolverService → InferenceEngine → TriangleValidator và RuleRegistry. Core không phụ thuộc giao diện. Mỗi lần giải có bản sao facts và danh sách trace riêng; không có dữ liệu suy luận được dùng chung giữa hai yêu cầu [2], [3].

3.2. Thuật toán suy diễn tiến

SOLVE(H, G):
  K ← bản sao H; Trace ← rỗng; kiểm tra K
  Nếu G đã có trong K: trả SOLVED
  Lặp từng lượt quét R01 … R46:
    Bỏ luật nếu thiếu đầu vào hoặc đã biết đầu ra
    Tính ứng viên y; nếu lỗi số học: ghi chẩn đoán, tiếp tục
  Tạo K′ = K hợp {y}; kiểm tra K′
    Nếu K′ mâu thuẫn: trả lỗi, không nhận ứng viên
    Nhận K ← K′; thêm bước vào Trace
    Nếu đã có đủ G: trả SOLVED
  Nếu hết một lượt không nhận biến mới: trả UNRESOLVED

Sau một bước thành công, engine tiếp tục phần còn lại của cùng lượt quét. Bộ kiểm tra chạy trước khi xét mục tiêu đã biết và trước khi nhận mỗi ứng viên; dữ kiện gốc không bị ghi đè. Mỗi bước thêm đúng một biến nên tối đa có $13 - |H|$ bước thành công. Với n biến, m luật và số đầu vào mỗi luật bị chặn, chi phí quét và kiểm tra là $O(nm)$; bộ nhớ facts/trace $O(n)$, ngoài kho luật $O(m)$.

Nếu phép tính gặp tràn/hụt số hoặc lỗi miền do giới hạn biểu diễn, engine ghi diagnostics và thử luật tiếp theo. Lỗi số học đơn lẻ không đủ chứng minh dữ kiện hình học vô nghiệm. Nếu ứng viên tính được xung đột với quan hệ đủ dữ kiện, ứng viên không được đưa vào facts hoặc trace.

3.3. Vết suy luận và giải thích phụ thuộc

Mỗi InferenceStep chứa step, rule_id, group, inputs kèm giá trị, output, value và formula. Trace đầy đủ giữ thứ tự phát sinh. Tùy chọn trace_targets ở API hoặc --trace-for ở CLI tạo relevant_trace bằng lần ngược các đầu ra cần thiết; số bước gốc được giữ nguyên, không thay thế trace đầy đủ.

Với H = {a = 3, b = 4, p = 6}, G = {hc}, chuỗi phụ thuộc rút gọn là R07 → R20 → R28, cho c = 5, S = 6, hc = 2,4. Chạy thực tế cùng dữ kiện với mục tiêu c, S, hc cho trace gồm R07, R15, R17, R19, R20, R22, R25, R28. Sự khác nhau là do engine còn tính góc và đường cao trung gian theo thứ tự luật; báo cáo không thay trace thực tế bằng đường phụ thuộc rút gọn.

3.4. Hợp đồng kết quả và giao diện

Bảng 3.2. Trạng thái nghiệp vụ

Trạng thái	Ý nghĩa
SOLVED	Đủ mục tiêu và vượt qua những kiểm tra hiện có; chưa chứng nhận toàn bộ cấu hình hình học tồn tại duy nhất.
UNRESOLVED	Chưa suy ra mục tiêu từ tập luật hoặc gặp giới hạn số học; không đồng nghĩa vô nghiệm toán học.
INVALID	Lỗi cấu trúc ở service/CLI, giá trị ngoài miền hoặc bộ cạnh vi phạm bất đẳng thức tam giác.
INCONSISTENT	Các dữ kiện riêng lẻ hợp lệ nhưng xung đột với quan hệ đã biết.

reason_code hỗ trợ xử lý độc lập với câu thông báo; diagnostics ghi lỗi hoặc giới hạn; stats ghi số bước, số biến đã biết, thời gian và lý do dừng. missing chỉ chứa mục tiêu chưa có trong facts, do đó có thể rỗng dù trạng thái INCONSISTENT khi mục tiêu đã có nhưng dữ kiện mâu thuẫn.

Giao diện đánh dấu kết quả cũ khi dữ kiện hoặc mục tiêu thay đổi và không hiển thị hình của lần giải cũ như kết quả mới. Yêu cầu bất đồng bộ cũ được bỏ qua khi đầu vào đã đổi. Tam giác SVG dùng các cạnh do backend suy ra; các trường hợp chưa đủ quy mô hoặc SSA được giải thích riêng, tránh tự chọn một nghiệm chưa được xác nhận.

CHƯƠNG 4. CÀI ĐẶT VÀ TRIỂN KHAI

4.1. Công nghệ và cấu trúc mã

Backend dùng Python 3.11+, Pydantic v2, FastAPI và Uvicorn. Frontend dùng React 19, TypeScript, Vite, Tailwind CSS và SVG. Phụ thuộc runtime và kiểm thử được tách ở requirements.txt/requirements-dev.txt; requirements-lock.txt mô tả môi trường đã kiểm tra. Không cần cơ sở dữ liệu hoặc dịch vụ đám mây.

Bảng 4.1. Các đường dẫn quan trọng (tính từ Bai1)

Đường dẫn	Nội dung
triangle_knowledge_base/app/knowledge/	Biến, công thức số học và danh mục luật.
triangle_knowledge_base/app/core/	Bộ suy diễn và validator.
triangle_knowledge_base/app/services/	Dịch vụ giải và lọc phụ thuộc.
triangle_knowledge_base/app/api/	Ứng dụng FastAPI và các endpoint.
triangle_knowledge_base/tests/	Kiểm thử backend và bản đối chiếu report_rules.csv.
frontend/src/; frontend/e2e/	Mã React, kiểm thử đơn vị/tích hợp và trình duyệt.
run_app.bat; run_app.sh; run_app.py	Launcher dùng chung vòng đời cho API và Vite.
docs/; Demo/	Hướng dẫn sử dụng, ảnh báo cáo và video trình duyệt thật.

4.2. Cài đặt và chạy local

Thực hiện tại Bai1 với Python 3.11+ và Node.js 20+ đã cài. Trên Windows, nếu python trỏ đến Microsoft Store thì thay bằng đường dẫn trình thông dịch thực tế. Môi trường kiểm tra dùng Python 3.13.5, Node.js 24.11.0 và npm 11.6.1.

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
cd frontend
npm install
cd ..
run_app.bat
```

Linux/macOS dùng .venv/bin/python và bash run_app.sh. Launcher ưu tiên môi trường .venv, kiểm tra dependencies và cổng 8000/5173, bật cả API và Vite; Ctrl+C dừng các tiến trình do launcher tạo. Script không tự cài thư viện. Nếu cần chạy riêng, dùng Uvicorn cho API và npm run dev trong frontend.

Giao diện: http://127.0.0.1:5173. Swagger: http://127.0.0.1:8000/docs. Vite chuyển /api đến backend theo VITE_BACKEND_URL, mặc định http://127.0.0.1:8000. VITE_API_BASE_URL mặc định rỗng để trình duyệt gọi cùng origin qua proxy.

4.3. CLI, REST API và thao tác sử dụng

```
python main.py --facts "a=3,b=4,c=5" --goals "p,S,hc,r,R"
python main.py --facts "a=3,b=4,p=6" --goals "hc" --trace-for "hc"
```

CLI có thể xuất JSON bằng --json. Mã thoát 0 tương ứng SOLVED, 1 tương ứng UNRESOLVED và 2 cho INVALID/INCONSISTENT hoặc lỗi cú pháp. Không nhập trùng tên biến. API nhận POST /api/v1/solve với facts, goals và trace_targets tùy chọn; GET /api/v1/variables, /api/v1/rules và /api/v1/rules/{id} cung cấp metadata.

API trả HTTP 422 cho request sai cấu trúc trước khi gọi service. Request hợp lệ về cấu trúc trả HTTP 200 cùng trạng thái nghiệp vụ. Tên lạ, boolean, chuỗi giả dạng số, NaN và infinity bị từ chối. Giao diện cho nhập dữ kiện, mở Advanced parameters khi cần, chọn mục tiêu rồi bấm Solve selected hoặc Calculate all possible. Người dùng có thể chọn kết quả để xem chuỗi phụ thuộc và highlight đại lượng trên hình.

CHƯƠNG 5. KIỂM THỬ VÀ KẾT QUẢ THỰC NGHIỆM

5.1. Môi trường và phương pháp kiểm tra

Các phép kiểm tra dưới đây được thực thi ngày 24/10/2026 trên mã tại commit 2ae25f2, Windows, Python 3.13.5. Bộ pytest kiểm tra mô hình dữ liệu, metadata, từng công thức trên hai tam giác, miền hình học, số học gần biên, trace, CLI và API. Vitest kiểm tra component và tích hợp với API thật; Playwright chạy trên Chromium. Chỉ ghi nhận những lệnh đã thực thi, không suy ra tỷ lệ bao phủ mã từ số test [4].

Bảng 5.1. Kết quả các bộ kiểm thử

Lệnh (từ thư mục tương ứng)	Kết quả thực tế
Bai1: python -m pytest -q	252 đạt; 1 cảnh báo deprecation của Starlette/HTTPX.
frontend: npm test	44 đạt trong 2 tệp kiểm thử.
frontend: npm run test:integration	9 đạt, gọi FastAPI thật qua HTTP.
frontend: npm run test:e2e	10 đạt trên Chromium.
frontend: npm run build	TypeScript và Vite build thành công.

Cảnh báo deprecation trong bộ backend không làm ca kiểm thử thất bại. E2E bao gồm dữ kiện SSS/SAS, thiếu dữ kiện, tam giác suy biến, dữ kiện mâu thuẫn, ba kích thước màn hình, thay đổi đầu vào và liên kết giữa kết quả với hình/trace. Kết quả này xác nhận phạm vi đã kiểm tra, không thay cho chứng minh đúng với mọi số thực.

5.2. Đối chiếu T01–T12

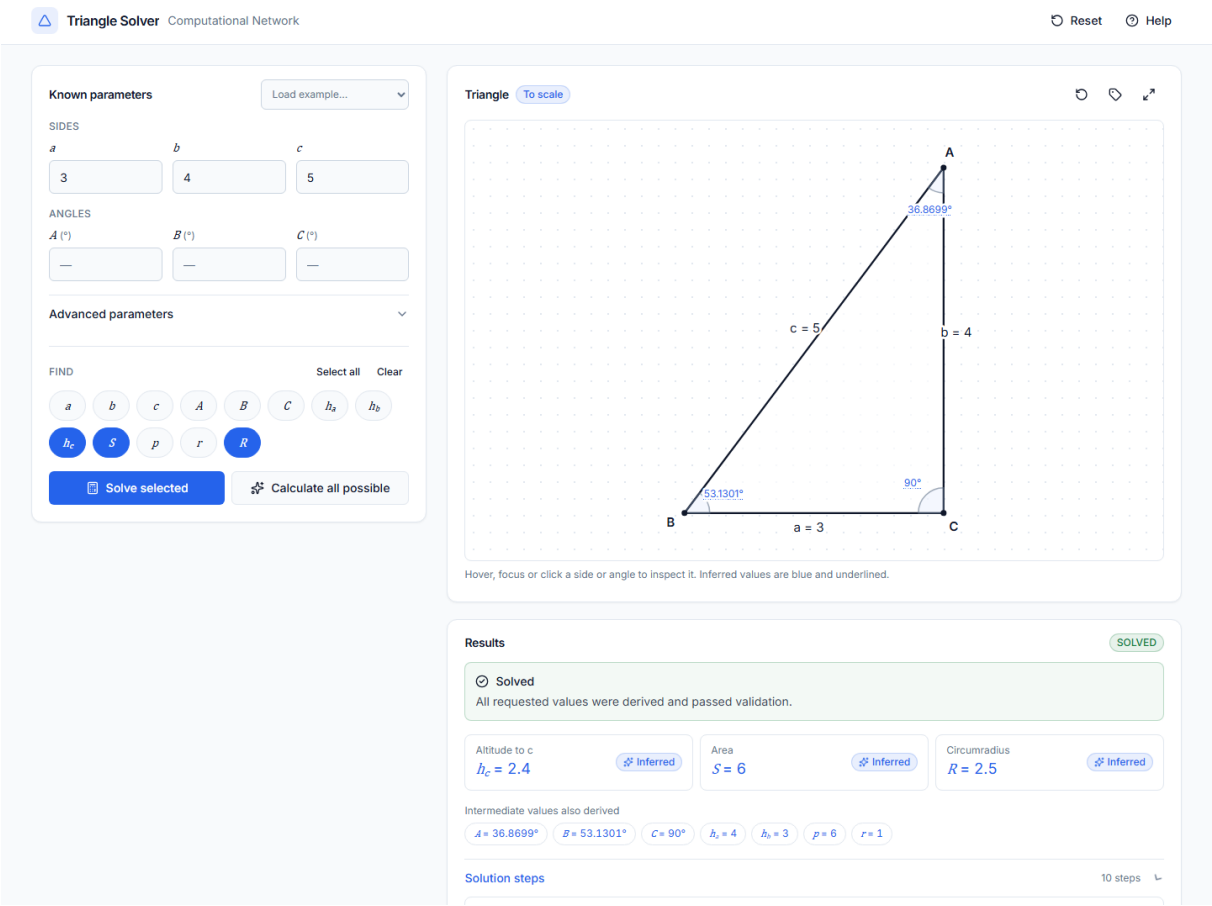
Bảng 5.2. Kết quả thực tế các ca trong báo cáo gốc

Mã	Dữ kiện → mục tiêu	Kết quả đối chiếu
T01	a=3, b=4, c=5 → p, S, hc, r, R	SOLVED; p=6; S=6; hc=2.4; r=1; R=2.5; đạt
T02	A=50, B=60, c=10 → C, a, b	SOLVED; C=70; a=8.15207; b=9.21605; đạt
T03	b=3, c=4, A=90 → a, S	SOLVED; a=5; S=6; đạt
T04	a=3, b=4, p=6 → c, S, hc	SOLVED; c=5; S=6; hc=2.4; đạt
T05	c=10, hc=6 → S	SOLVED; S=30; đạt

Mã	Dữ kiện → mục tiêu	Kết quả đối chiếu
T06	a=3 → S	UNRESOLVED; INSUFFICIENT_FACTS; đạt
T07	a=1, b=2, c=3 → S	INVALID; INVALID_TRIANGLE; đạt
T08	a=-3, b=4, c=5 → S	INVALID; INVALID_INPUT; đạt
T09	a=3, b=4, c=5, S=7 → S	INCONSISTENT; CONFLICTING_FACTS; đạt
T10	A=30, B=60, C=90 → a	UNRESOLVED; INSUFFICIENT_FACTS; đạt
T11	A=30, a=5, b=8 → B, C, c	UNRESOLVED; INSUFFICIENT_FACTS; đạt
T12	S=6, p=6 → r	SOLVED; r=1; đạt

T09 cho INCONSISTENT dù mục tiêu S đã có: giá trị S = 7 được giữ nguyên và bị đối chiếu với Heron cho S = 6. T11 không có luật arcsin để tính B từ sin B = 0,8; hai khả năng góc là khoảng 53,130102° và 126,869898°, nên hệ thống không tự chọn. Các giá trị làm tròn trong bảng chỉ phục vụ đọc báo cáo; phép kiểm tra dùng sai số đã nêu ở mục 2.4.

5.3. Minh họa ứng dụng thực tế



Hình 5.1. Kết quả nhập a = 3, b = 4, c = 5; mục tiêu S, hc, R trên ứng dụng local ngày 24/10/2026

Hình 5.1 là ảnh Playwright chụp sau khi kiểm tra SOLVED, S = 6, hc = 2,4 và R = 2,5 trên trình duyệt thật. Khu vực Solution steps thể hiện luật backend; hình tam giác được dựng bằng SVG. Video Demo/triangle-solver-demo.webm là bản ghi có sẵn khoảng 87 giây, không âm thanh; ảnh báo cáo mới không giả lập video hoặc số liệu.

5.4. Đánh giá và giới hạn thực nghiệm

Các ca T01–T12 khớp kỳ vọng trong báo cáo gốc. Việc kiểm tra ứng viên trước khi nhận và kiểm tra mục tiêu đã biết giúp tránh trả kết quả thành công trên dữ kiện mâu thuẫn. Lọc trace làm rõ lý do một kết quả mà vẫn bảo toàn toàn bộ lịch sử. Chưa đo hiệu năng theo nhiều cấu hình phần cứng; thời gian stats thay đổi giữa các lần chạy nên không dùng một giá trị đơn lẻ để kết luận tốc độ hệ thống.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Bài 1 đã triển khai CSTT đúng danh mục 13 biến, 10 nhóm và 46 luật; suy diễn tiến có tính dừng, kiểm tra dữ kiện trước/sau suy luận, giữ dữ kiện gốc và giải thích bằng trace. CLI và API dùng chung dịch vụ, giao diện trực quan hóa kết quả từ backend. Các bộ kiểm thử đã chạy thành công trong môi trường nêu ở Chương 5.

6.2. Hạn chế

Không suy ngược Heron, không giải hệ ký hiệu tổng quát, không phân nhánh SSA/arcsin và không tối ưu số bước. Validator chỉ chứng minh được những vi phạm khi đủ dữ kiện tương ứng; SOLVED không bảo đảm nghiệm hình học duy nhất. Số học float có giới hạn tràn/hụt và độ chính xác; dữ liệu cực đoan cần chuẩn hóa đơn vị. Kiểm thử trình duyệt hiện giới hạn ở Chromium.

6.3. Hướng phát triển

Có thể bổ sung bộ giải nhiều nghiệm có biểu diễn nhánh rõ ràng, chuẩn hóa đơn vị và chẩn đoán sai số, lựa chọn đường suy luận theo chi phí và kiểm thử trên nhiều trình duyệt. Mỗi mở rộng phải cập nhật danh mục luật, điều kiện áp dụng và ca kiểm thử tương ứng; không thêm chiều tính chỉ vì có thể biến đổi đại số trên giấy.

TÀI LIỆU THAM KHẢO

[1] Nguyễn Đình Hiển. Các hệ cơ sở tri thức, Chương 3, mục 3.3.3: Mạng tính toán. Thông tin thư mục kế thừa báo cáo gốc: “4b. Phuong phap BDTT (tt).pdf”, slide 10–25; tệp bài giảng không kèm repository.

[2] Nhóm CS217. Mã nguồn Bài 1, app/knowledge/rules.py, app/knowledge/formulas.py và app/models/. Repository CS217-BT2, commit 2ae25f2, đối chiếu 24/10/2026.

[3] Nhóm CS217. Mã nguồn app/core/inference_engine.py, app/core/validator.py và app/services/. Cùng repository và phiên bản với [2].

[4] Nhóm CS217. Bai1/README.md, docs/USER_GUIDE.md, Demo/README.md và bộ kiểm thử triangle_knowledge_base/tests/, frontend/src/, frontend/e2e/. Đối chiếu 24/10/2026.

PHỤ LỤC A. DANH MỤC 46 LUẬT CÓ HƯỚNG

Danh mục được lấy trực tiếp từ RuleRegistry tại phiên bản đã kiểm tra và đối chiếu với report_rules.csv. sin_deg, cos_deg, acos_deg làm việc theo độ. Mọi luật yêu cầu đầu vào hữu hạn, đúng miền; kiểm tra ràng buộc được trình bày ở mục 2.4. Các biểu thức dưới đây là metadata hiển thị, không phải chuỗi được eval.

Bảng A.1. Danh mục luật R01–R46

Mã / nhóm	Đầu vào → đầu ra	Công thức
R01 / Q01	B, C → A	A = 180 - B - C
R02 / Q01	A, C → B	B = 180 - A - C
R03 / Q01	A, B → C	C = 180 - A - B
R04 / Q02	a, b, c → p	p = (a + b + c) / 2
R05 / Q02	p, b, c → a	a = 2*p - b - c
R06 / Q02	p, a, c → b	b = 2*p - a - c
R07 / Q02	p, a, b → c	c = 2*p - a - b
R08 / Q03	b, A, B → a	a = b*sin_deg(A)/sin_deg(B)
R09 / Q03	c, A, C → a	a = c*sin_deg(A)/sin_deg(C)
R10 / Q03	a, B, A → b	b = a*sin_deg(B)/sin_deg(A)
R11 / Q03	c, B, C → b	b = c*sin_deg(B)/sin_deg(C)
R12 / Q03	a, C, A → c	c = a*sin_deg(C)/sin_deg(A)
R13 / Q03	b, C, B → c	c = b*sin_deg(C)/sin_deg(B)
R14 / Q04	b, c, A → a	a = sqrt(b^2 + c^2 - 2*b*c*cos_deg(A))
R15 / Q04	a, b, c → A	A = acos_deg((b^2 + c^2 - a^2)/(2*b*c))
R16 / Q04	a, c, B → b	b = sqrt(a^2 + c^2 - 2*a*c*cos_deg(B))
R17 / Q04	a, b, c → B	B = acos_deg((a^2 + c^2 - b^2)/(2*a*c))
R18 / Q04	a, b, C → c	c = sqrt(a^2 + b^2 - 2*a*b*cos_deg(C))
R19 / Q04	a, b, c → C	C = acos_deg((a^2 + b^2 - c^2)/(2*a*b))
R20 / Q05	p, a, b, c → S	S = sqrt(p*(p-a)*(p-b)*(p-c))
R21 / Q06	a, ha → S	S = a*ha/2
R22 / Q06	S, a → ha	ha = 2*S/a

Mã / nhóm	Đầu vào → đầu ra	Công thức
R23 / Q06	S, ha → a	a = 2*S/ha
R24 / Q06	b, hb → S	S = b*hb/2
R25 / Q06	S, b → hb	hb = 2*S/b
R26 / Q06	S, hb → b	b = 2*S/hb
R27 / Q06	c, hc → S	S = c*hc/2
R28 / Q06	S, c → hc	hc = 2*S/c
R29 / Q06	S, hc → c	c = 2*S/hc
R30 / Q07	b, c, A → S	S = b*c*sin_deg(A)/2
R31 / Q07	a, c, B → S	S = a*c*sin_deg(B)/2
R32 / Q07	a, b, C → S	S = a*b*sin_deg(C)/2
R33 / Q08	p, r → S	S = p*r
R34 / Q08	S, p → r	r = S/p
R35 / Q08	S, r → p	p = S/r
R36 / Q09	a, b, c, R → S	S = a*b*c/(4*R)
R37 / Q09	a, b, c, S → R	R = a*b*c/(4*S)
R38 / Q09	S, R, b, c → a	a = 4*S*R/(b*c)
R39 / Q09	S, R, a, c → b	b = 4*S*R/(a*c)
R40 / Q09	S, R, a, b → c	c = 4*S*R/(a*b)
R41 / Q10	R, A → a	a = 2*R*sin_deg(A)
R42 / Q10	a, A → R	R = a/(2*sin_deg(A))
R43 / Q10	R, B → b	b = 2*R*sin_deg(B)
R44 / Q10	b, B → R	R = b/(2*sin_deg(B))
R45 / Q10	R, C → c	c = 2*R*sin_deg(C)
R46 / Q10	c, C → R	R = c/(2*sin_deg(C))